Table des matières

Labo 05 — Tester, historiser et documenter 1
Avant de commencer 2
Étape 1 — Les tests génériques des dimensions 3
Étape 2 — Le test qui passe au vert sur un désastre 4
Étape 3 — Les tests du fait 4
Étape 4 — Le test singulier, et la jointure qui ne voit rien 5
Étape 5 — La sévérité et le seuil 7
Étape 6 — Le snapshot, et deux ans d'histoire rejoués 8
Étape 7 — La dimension en type 2 10
Étape 8 — Le fait sur la version d'époque 12
Étape 9 — Le test unitaire de la jointure temporelle 14
Étape 10 — La documentation 16
Étape 11 — Déclarer qui consomme 17
Étape 12 — Le résumé du rapport, et l'arbitrage 17
Récapitulatif 19
Dépannage 20

Labo 05 — Tester, historiser et documenter

Labo · Cours 5/9 — Formation Analytics Engineering

Ce que vous allez faire. Poser vingt-quatre tests sur l'étoile du Cours 04, dont un qui passera au vert sur un désastre et vous apprendra pourquoi. Puis lui rendre son histoire : un snapshot qui rejoue deux ans de révisions de paliers, une dimension en type 2, et un fait qui joint la version d'époque au lieu de recalculer sa clé. Vous casserez cette jointure deux fois avant de la réussir, et chacun des deux contrôles du Cours 04 en attrapera un. Objectif de fin : `dbt build` monte l'étoile et vérifie vingt-quatre choses en deux secondes, et le chiffre d'affaires par palier de 2023 a changé sans qu'un cent ne bouge du total.

Deux terminaux servent dans ce labo. L'Anaconda Prompt, environnement `formations` actif, sert à dbt et à Git ; il est ouvert dans `Projet_Boreal/02_ae`. La ligne de commande DuckDB sert à vérifier ce que dbt a écrit, et elle doit être fermée quand dbt travaille.

La liste de contrôle du cours :

© 2026 iAlexMG. Tous droits réservés.

- ☐ Les tests génériques des trois dimensions, au vert
- ☐ `accepted_values` constaté au vert sur un décodage entièrement raté
- ☐ Les tests de relation du fait vers ses trois dimensions
- ☐ Le test singulier de réconciliation, en jointure externe complète
- ☐ Le seuil sur `annee_modele` : 23 passe, 24 avertirait
- ☐ Le snapshot rejoué sur quatre lots : 205 lignes, dont 49 versions fermées
- ☐ `dim_concessionnaire` en type 2 : 205 versions, 156 courantes
- ☐ `fct_ventes` sur la version d'époque : 75 652 lignes, 0,00 $ d'écart
- ☐ Le test unitaire de la jointure temporelle, aux trois bornes
- ☐ `dbt docs generate`, et le `manifest.json` lu
- ☐ L'`exposure` au nom de l'équipe de Business Intelligence
- ☐ `rpt_ventes_par_palier`, et l'arbitrage écrit

Le labo se fait en deux séances. La première va jusqu'à l'étape 5, la seconde porte l'histoire, la documentation et la restitution.

Avant de commencer

Ce labo part du projet tel que le Cours 04 l'a laissé. Vérifiez-le dans l'Anaconda Prompt, depuis `Projet_Boreal/02_ae` :

```
dbt debug
dbt build
```

`dbt build` doit rendre `PASS=10 WARN=0 ERROR=0` : cinq vues de staging, un modèle intermédiaire invisible, et les quatre tables de l'étoile. Aucun test encore, c'est l'objet du jour.

Fermez ensuite la ligne de commande DuckDB si elle est ouverte, par `.exit`.

© 2026 iAlexMG. Tous droits réservés.

Étape 1 — Les tests génériques des dimensions

Théorie : `Theorie_05_AE.md`, sections 1 et 2.

Ce fichier portera les descriptions et les tests de la couche, et il grossira jusqu'à la fin du labo. Créez-le, dans `models/marts/_marts.yml` :

```yaml
version: 2

models:
  - name: dim_date
    description: "Une ligne par jour du calendrier, du 2022-01-01 au 2026-03-31."
    columns:
      - name: date_jour
        description: "La clé de la dimension : la date elle-même."
        data_tests:
          - unique
          - not_null
      - name: nom_mois
        data_tests:
          - not_null

  - name: dim_produit
    description: "Une ligne par article. Aucune historisation."
    columns:
      - name: cle_produit
        data_tests:
          - unique
          - not_null
      - name: gamme
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: ['Motoneige', 'Motomarine', 'Hors-route', 'Pièces et accessoires']
```

Deux points de forme méritent un mot, parce qu'ils ont changé récemment et que les exemples anciens traînent partout. Le bloc s'appelle `data_tests:` et non `tests:`. Et les paramètres d'un test paramétré vont sous `arguments:` ; sans lui, dbt fonctionne mais affiche un avertissement de dépréciation à chaque commande.

```
dbt test --select dim_date dim_produit
```

Attendu : sept tests, tous au vert. La sélection prend aussi, plus tard dans le labo, les tests qui pointent vers ces modèles sans leur appartenir, comme les relations du fait.

© 2026 iAlexMG. Tous droits réservés.

Remarquez au passage le nom que dbt donne à chacun, par exemple `accepted_values_dim_produit_gamme__Motoneige__Motomarine__Hors_route__Pièces_et_accessoires`. Il n'est pas beau et il est précieux : quand un test échoue dans une exécution de quarante, son nom dit à lui seul quelle colonne de quel modèle a fauté.

Étape 2 — Le test qui passe au vert sur un désastre

Théorie : `Theorie_05_AE.md`, section 2.

Cette étape ne construit rien. Elle vous fait constater une chose qu'il vaut mieux avoir vue une fois de ses yeux que lue dans une liste de bonnes pratiques.

Ouvrez `models/staging/stg_articles.sql` et sabotez le décodage de la gamme : remplacez les quatre codes `'MN'`, `'MM'`, `'HR'` et `'PAC'` par `'ZZ'`, `'ZY'`, `'ZX'` et `'ZW'`, qui n'existent pas.

```
dbt run --select stg_articles dim_produit
dbt test --select dim_produit
```

Deux lignes comptent dans la sortie :

```
PASS accepted_values_dim_produit_gamme__Motoneige__Motomarine__Hors_route__Pièces_et_accessoires
FAIL 68 not_null_dim_produit_gamme
```

Le décodage a entièrement raté, les 68 articles ont une gamme vide, et le test qui vérifie les valeurs acceptées passe au vert. Il ne ment pas vraiment : il vérifie que ce qui est là fait partie de la liste, et il n'y a rien. C'est le `not_null` posé à côté qui sauve la mise.

Remettez les quatre codes en place, et reconstruisez :

```
dbt run --select stg_articles dim_produit
dbt test --select dim_produit
```

Les deux tests repassent au vert. Retenez la règle, elle n'a pas d'exception : un `accepted_values` sans `not_null` à côté ne vérifie rien.

Étape 3 — Les tests du fait

Théorie : `Theorie_05_AE.md`, section 2.

Le fait recalcule les clés de ses dimensions au lieu de les joindre, et le Cours 04 a dit ce que cela laissait ouvert. Voici la fermeture.

Ajoutez à `models/marts/_marts.yml` :

© 2026 iAlexMG. Tous droits réservés.

```yaml
  - name: fct_ventes
    description: "Le fait des ventes, au grain de la ligne de commande, hors postes refusés."
    data_tests:
      - dbt_utils.unique_combination_of_columns:
          arguments:
            combination_of_columns:
              - numero_commande
              - numero_poste
    columns:
      - name: cle_date
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_jour
      - name: cle_produit
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_produit')
                field: cle_produit
      - name: valeur_nette
        data_tests:
          - not_null
```

Le premier test n'est pas posé sur une colonne mais sur le modèle : c'est le contrôle du grain du Cours 04, celui que vous lanciez à la main, écrit une fois pour toutes.

```
dbt test --select fct_ventes
```

Attendu : six tests au vert. La dimension du concessionnaire n'est pas encore dans la liste, parce qu'elle va changer de forme à l'étape 7 ; son test de relation viendra avec.

Étape 4 — Le test singulier, et la jointure qui ne voit rien

Théorie : `Theorie_05_AE.md`, section 3.

La réconciliation du Cours 04 devient un test qui se relance tout seul. Dans `tests/assert_reconciliation_ventes.sql` :

© 2026 iAlexMG. Tous droits réservés.

```sql
with fait as (

    select numero_commande, sum(valeur_nette) as valeur
    from {{ ref('fct_ventes') }}
    group by 1

),

erp as (

    select numero_commande, valeur_nette as valeur
    from {{ ref('stg_commandes') }}
    where valeur_nette <> 0

)

select
    coalesce(fait.numero_commande, erp.numero_commande) as numero_commande,
    fait.valeur as valeur_du_fait,
    erp.valeur as valeur_de_l_erp

from fait
full outer join erp on erp.numero_commande = fait.numero_commande

where fait.numero_commande is null
   or erp.numero_commande is null
   or abs(fait.valeur - erp.valeur) > 0.005
```

Trois choses se lisent dans cette requête.

Le `full outer join` est le cœur du test, et il n'est pas négociable. Une jointure interne ne compare que ce qui existe des deux côtés : un document absent du fait n'entre pas dans la comparaison, ne produit aucune ligne, et le test passe. Vous pouvez le constater en une requête, dans DuckDB, sur un fait amputé de janvier 2022 :

```sql
create or replace temp view fait_ampute as
select numero_commande, sum(valeur_nette) as valeur
from marts.fct_ventes
where cle_date >= date '2022-02-01'
group by 1;

select
    (select count(*) from fait_ampute f
       join staging.stg_commandes e on e.numero_commande = f.numero_commande
```


© 2026 iAlexMG. Tous droits réservés.

```
  where abs(f.valeur - e.valeur_nette) > 0.005) as vu_par_la_jointure_interne,
(select count(*) from fait_ampute f
   full outer join (select * from staging.stg_commandes where valeur_nette <> 0) e
     on e.numero_commande = f.numero_commande
  where f.numero_commande is null or e.numero_commande is null
     or abs(f.valeur - e.valeur_nette) > 0.005) as vu_par_la_jointure_externe;
```

Attendu : 0 et 518. La jointure interne ne voit rien du tout ; l'externe voit les 518 commandes disparues.

Renvoi : le `full outer join` et le rapprochement de deux sources sont au Cours 01 de SQL, section 7 ; la comparaison dans les deux sens, à son Cours 07, section 6.

Le filtre `valeur_nette <> 0` écarte les 227 commandes entièrement refusées, dont l'ERP porte une valeur nette nulle et que le fait ne contient pas. Sans lui, le test échouerait tous les jours sur un écart qui n'en est pas un.

Et la tolérance de `0.005` compare des décimaux sans se faire piéger par le dernier cent.

```
dbt test --select assert_reconciliation_ventes
```

Attendu : au vert, 0 ligne rendue.

Étape 5 — La sévérité et le seuil

Théorie : `Theorie_05_AE.md`, section 4.

Chez Boréal, 23 articles n'ont pas d'année-modèle : ce sont les pièces et accessoires. Un `not_null` sur cette colonne échouerait tous les jours pour rien, et on finirait par le retirer.

Ajoutez plutôt un seuil, sous `dim_produit` dans `_marts.yml` :

```
      - name: annee_modele
        description: "Vide pour les 23 pièces et accessoires, ce qui est normal."
        data_tests:
          - not_null:
              config:
                severity: warn
                warn_if: ">23"
                error_if: ">30"
```

```
dbt test --select dim_produit
```


© 2026 iAlexMG. Tous droits réservés.

Le test passe, en silence, alors que 23 lignes le font échouer au sens strict. Vérifiez que le seuil mord vraiment en le descendant à `">22"` le temps d'une exécution :

```
WARN 23 not_null_dim_produit_annee_modele
[WARNING]: Got 23 results, configured to warn if >22
```

Remettez `">23"`. Ce que vous venez d'écrire n'est pas un test assoupli, c'est une affirmation sur les données, datée et versionnée : il y en a 23, et nous voulons le savoir si ça change.

Étape 6 — Le snapshot, et deux ans d'histoire rejoués

Théorie : `Theorie_05_AE.md`, section 6.

Ici commence la seconde séance.

La couche `RAW` garde les 49 extraits mensuels du référentiel, donc l'histoire des paliers y est, étalée dans 7 481 lignes. Un système source ordinaire ne l'aurait pas gardée, et c'est pour ce cas ordinaire que le snapshot existe. Chez Boréal, il va servir à rejouer deux ans en quatre commandes.

Créez `snapshots/snp_concessionnaire.sql` :

```
{% snapshot snp_concessionnaire %}

{{
    config(
        target_schema='snapshots',
        unique_key='code_concessionnaire',
        strategy='check',
        check_cols=['palier'],
        updated_at='date_extrait'
    )
}}

select
    kunnr as code_concessionnaire,
    case kukla
        when '01' then 'Or'
        when '02' then 'Argent'
        when '03' then 'Bronze'
    end as palier,
    (date_trunc('month', strptime(left(_lot, 7), '%Y-%m'))
        + interval 1 month - interval 1 day)::date as date_extrait
```


© 2026 iAlexMG. Tous droits réservés.

```
from {{ source('raw', 'erp_kna1') }}

{% if var('lot', none) %}
where _lot = '{{ var('lot') }}'
{% else %}
where _lot = (select max(_lot) from {{ source('raw', 'erp_kna1') }})
{% endif %}

{% endsnapshot %}
```

Quatre choses à remarquer, et la dernière est celle qui évite une heure de dépannage.

`strategy='check'` avec `check_cols=['palier']` : dbt ouvre une version quand le palier change, et seulement alors.

`updated_at='date_extrait'` : sans cette ligne, dbt daterait chaque version de l'heure du lancement, et votre histoire serait datée d'aujourd'hui. Avec elle, elle est datée de la fin du mois de l'extrait.

Le `left(_lot, 7)` traite le lot `2023-03_bis`, qui n'est pas un nom de mois, et le ramène à son mois.

Et le `var('lot', none)` porte une valeur par défaut. Un `var()` sans défaut fait échouer toute commande qui compile le projet, `dbt docs generate` comprise, sur `Required var 'lot' not found in config`. Avec le défaut, une exécution sans option capture l'état courant, ce qu'un ordonnanceur ferait chaque nuit.

Rejouez maintenant quatre moments de l'histoire, dans l'ordre :

```
dbt snapshot --vars "{lot: 2022-01}"
dbt snapshot --vars "{lot: 2024-06}"
dbt snapshot --vars "{lot: 2024-07}"
dbt snapshot --vars "{lot: 2025-12}"
```

Puis, dans DuckDB :

```
select
    count(*) as lignes,
    count(distinct code_concessionnaire) as concessionnaires,
    count(*) filter (where dbt_valid_to is null) as versions_ouvertes,
    count(*) filter (where dbt_valid_to is not null) as versions_fermees
from snapshots.snp_concessionnaire;

select dbt_valid_from, dbt_valid_to, count(*) as lignes
from snapshots.snp_concessionnaire
group by 1, 2
order by 1, 2;
```


© 2026 iAlexMG. Tous droits réservés.

Mesure	Attendu
Lignes	205
Concessionnaires	156
Versions ouvertes	156
Versions fermées	49

Les 49 versions fermées le sont toutes au 31 juillet 2024 : c'est la révision des paliers, et vous venez de la retrouver sans jamais l'avoir cherchée.

Deux honnêtetés avant de continuer. Les quatre lots suffisent pour les paliers, parce que le seul changement de palier de l'histoire est celui de juillet 2024 ; ils datent en revanche l'ouverture de cinq concessionnaires au lot où le snapshot les a vus pour la première fois, et non à leur vrai mois. Rejouer les 49 lots corrigerait ces cinq dates, pour 49 commandes et cinq minutes.

Et le changement lui-même a eu lieu le 1er juillet, pas le 31 : l'extrait mensuel ne le montre qu'à la fin du mois. Les ventes de juillet 2024 chez ces 49 concessionnaires, 4 314 525,62 $, seront donc attribuées à l'ancien palier. Un snapshot n'est pas plus précis que la fréquence à laquelle il regarde.

Étape 7 — La dimension en type 2

Théorie : `Theorie_05_AE.md`, section 7.

Réécrivez `models/marts/dim_concessionnaire.sql` en entier :

```
with versions as (

    select
        code_concessionnaire,
        palier,
        dbt_valid_from,
        dbt_valid_to,
        row_number() over (
            partition by code_concessionnaire
            order by dbt_valid_from
        ) as rang

    from {{ ref('snp_concessionnaire') }}

)
```


© 2026 iAlexMG. Tous droits réservés.

```sql
select
    {{ dbt_utils.generate_surrogate_key(['v.code_concessionnaire', 'v.dbt_valid_from']) }}
        as cle_concessionnaire,
    v.code_concessionnaire,
    c.nom_concessionnaire,
    c.ville,
    c.province,
    r.nom_province,
    v.palier,
    case when v.rang = 1 then date '1900-01-01' else v.dbt_valid_from end as valide_de,
    v.dbt_valid_to as valide_jusqua,
    v.dbt_valid_to is null as est_courante,
    c.date_creation,
    c.est_supprime

from versions as v
inner join {{ ref('stg_concessionnaires') }} as c
    on c.code_concessionnaire = v.code_concessionnaire
left join {{ ref('stg_regions') }} as r
    on r.code_province = c.province
```

Trois décisions sont écrites là-dedans, et il faut pouvoir les défendre.

La clé se hache sur le code et la date d'ouverture, et non sur le seul code. Deux versions du même concessionnaire auraient sinon la même clé.

Le palier vient du snapshot, le reste vient du staging. Autrement dit le palier est historisé et le nom ne l'est pas : une version fermée affiche le nom d'aujourd'hui. C'est un choix, il se documente, et il se change le jour où quelqu'un a besoin de l'historique des noms.

La première version de chaque concessionnaire s'ouvre au 1er janvier 1900. Sans ce plancher, une vente antérieure au premier regard du snapshot ne trouverait aucune version, et disparaîtrait. L'étape suivante vous le fera mesurer.

Ajoutez les tests dans `_marts.yml`, sous les modèles :

```yaml
  - name: dim_concessionnaire
    description: "Une ligne par version de concessionnaire. Le palier est suivi en type 2."
    data_tests:
      - dbt_utils.unique_combination_of_columns:
          arguments:
            combination_of_columns:
              - code_concessionnaire
              - valide_de
    columns:
      - name: cle_concessionnaire
```


© 2026 iAlexMG. Tous droits réservés.

```
        description: "Clé de substitution de la VERSION, hachée sur le code et la date d'ouverture."
        data_tests:
          - unique
          - not_null
      - name: palier
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: ['Or', 'Argent', 'Bronze']
```

```
dbt run --select dim_concessionnaire
```

```
select
    count(*) as versions,
    count(distinct code_concessionnaire) as concessionnaires,
    count(*) filter (where est_courante) as courantes,
    count(*) filter (where not est_courante) as fermees
from marts.dim_concessionnaire;
```

Attendu : 205 versions, 156 concessionnaires, 156 courantes, 49 fermées.

Étape 8 — Le fait sur la version d'époque

Théorie : `Theorie_05_AE.md`, section 8.

Le fait ne peut plus recalculer la clé du concessionnaire : le code ne désigne plus une ligne, il en désigne une ou deux selon la date. Il faut joindre.

Vous allez écrire cette jointure trois fois. Les deux premières sont fausses, et c'est voulu : les deux contrôles du Cours 04 vont attraper chacun un défaut, et vous verrez pourquoi il en faut deux.

Première version, dans `models/marts/fct_ventes.sql`, avec un intervalle fermé des deux côtés :

```
select
    l.date_commande as cle_date,
    d.cle_concessionnaire,
    {{ dbt_utils.generate_surrogate_key(['l.code_article']) }} as cle_produit,
    l.numero_commande,
    l.numero_poste,
    l.quantite,
    l.valeur_nette,
```


© 2026 iAlexMG. Tous droits réservés.

```
    l.cout,
    l.marge

from {{ ref('int_lignes_enrichies') }} as l
inner join {{ ref('dim_concessionnaire') }} as d
    on d.code_concessionnaire = l.code_concessionnaire
   and l.date_commande >= d.valide_de
   and l.date_commande <= coalesce(d.valide_jusqua, date '9999-12-31')
```

```
dbt run --select fct_ventes
```

```
select
    count(*) as lignes,
    count(distinct (numero_commande, numero_poste)) as couples,
    sum(valeur_nette) as valeur_nette,
    sum(valeur_nette) - (select sum(valeur_nette) from staging.stg_commandes) as ecart
from marts.fct_ventes;
```

Attendu : 73 774 lignes pour 73 758 couples, 420 707 489,69 $, et un écart de -9 440 522,47 $. Deux contrôles tombent d'un coup, et ils signalent deux défauts différents.

Le grain d'abord. dbt ferme une version à la date où la suivante s'ouvre : le 31 juillet 2024 est à la fois la fin de l'ancienne et le début de la neuve, donc une vente de ce jour-là correspond aux deux. Cinquante-six lignes de commande portent cette date, et seize se dédoublent.

Corrigez l'intervalle, semi-ouvert à la fin :

```
    and (d.valide_jusqua is null or l.date_commande < d.valide_jusqua)
```

Relancez le modèle et la même vérification. Le grain est réparé, 73 758 lignes pour 73 758 couples, et l'écart est toujours là, à -9 455 090,05 $. C'est le second défaut, que le contrôle du grain ne pouvait pas voir : 1 894 lignes ne trouvent aucune version. Il y en a 1 418 en janvier 2022, parce que le snapshot n'a regardé pour la première fois que le 31 janvier ; les 476 autres appartiennent aux six concessionnaires ouverts après ce premier lot, que le snapshot n'a vus qu'au lot suivant.

C'est le plancher de l'étape 7 qui les récupère. Si vous l'avez écrit, tout est déjà en place : vérifiez que votre `dim_concessionnaire` porte bien `case when v.rang = 1 then date '1900-01-01' else v.dbt_valid_from end`, reconstruisez les deux modèles, et refaites la vérification.

```
dbt run --select dim_concessionnaire fct_ventes
```

Mesure	Attendu
Lignes	75 652
Couples distincts	75 652

© 2026 iAlexMG. Tous droits réservés.

Mesure	Attendu
Valeur nette	430 148 012,16 $
Écart avec l'ERP	0,00 $

L'étoile est revenue exactement où elle était, et c'est le résultat qui compte : l'historisation change l'attribution de la valeur, jamais son total.

Ajoutez enfin le test de relation qui manquait, sous `fct_ventes` dans `_marts.yml` :

```
      - name: cle_concessionnaire
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_concessionnaire')
                field: cle_concessionnaire
```

Et regardez ce qui a changé pour le métier :

```
select c.palier, count(*) as lignes, sum(f.valeur_nette) as valeur_nette
from marts.fct_ventes f
join marts.dim_concessionnaire c on c.cle_concessionnaire = f.cle_concessionnaire
join marts.dim_date d on d.date_jour = f.cle_date
where d.annee = 2023
group by 1
order by 1;
```

Palier	Version d'époque	Ce que le Cours 04 disait
Argent	42 450 461,33 $	43 848 074,19 $
Bronze	27 732 309,27 $	24 452 220,42 $
Or	38 323 046,25 $	40 205 522,24 $

Sur toute l'histoire, 17 733 lignes changent de palier selon la lecture, pour 94 628 792,88 $. Les deux tableaux sont justes ; il faut savoir lequel on publie, et le dire.

Étape 9 — Le test unitaire de la jointure temporelle

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_AE.md`, section 5.

La jointure que vous venez d'écrire a trois bornes qui comptent : une vente avant le changement, une vente le jour même, une vente avant la première version. Aucun test de données ne les vérifie, parce qu'il faudrait que les trois cas existent dans les données, et parce qu'un test de données regarde le résultat, pas la logique.

Ajoutez, à la fin de `_marts.yml`, hors du bloc `models:` :

```yaml
unit_tests:
  - name: test_version_d_epoque
    description: "La jointure temporelle choisit la bonne version, aux trois bornes qui comptent."
    model: fct_ventes
    given:
      - input: ref('int_lignes_enrichies')
        format: sql
        rows: |
          select 'C1' as numero_commande, '10' as numero_poste,
                 cast('2024-06-15' as date) as date_commande,
                 'D1' as code_concessionnaire, 'A1' as code_article,
                 cast(1 as decimal(18, 3)) as quantite,
                 cast(100 as decimal(18, 2)) as valeur_nette,
                 cast(60 as decimal(18, 2)) as cout,
                 cast(40 as decimal(18, 2)) as marge
          union all
          select 'C2', '10', cast('2024-07-31' as date), 'D1', 'A1',
                 cast(1 as decimal(18, 3)), cast(100 as decimal(18, 2)),
                 cast(60 as decimal(18, 2)), cast(40 as decimal(18, 2))
          union all
          select 'C3', '10', cast('2022-01-05' as date), 'D1', 'A1',
                 cast(1 as decimal(18, 3)), cast(100 as decimal(18, 2)),
                 cast(60 as decimal(18, 2)), cast(40 as decimal(18, 2))
      - input: ref('dim_concessionnaire')
        rows:
          - {cle_concessionnaire: 'ancienne', code_concessionnaire: 'D1',
             valide_de: '1900-01-01', valide_jusqua: '2024-07-31'}
          - {cle_concessionnaire: 'neuve', code_concessionnaire: 'D1',
             valide_de: '2024-07-31', valide_jusqua: null}
    expect:
      rows:
        - {numero_commande: 'C1', cle_concessionnaire: 'ancienne'}
        - {numero_commande: 'C2', cle_concessionnaire: 'neuve'}
        - {numero_commande: 'C3', cle_concessionnaire: 'ancienne'}
```


© 2026 iAlexMG. Tous droits réservés.

La première entrée est donnée en `format: sql` et la seconde en lignes ordinaires, et ce n'est pas un caprice de style. Pour préparer une entrée, dbt va chercher les types des colonnes dans la base ; `int_lignes_enrichies` est `ephemeral`, donc il n'y a aucun objet où les chercher, et le test s'arrête sur `Not able to get columns for unit test … because the relation doesn't exist`. Un `select` qui déclare ses propres types règle la question.

Remarquez aussi que `expect` ne cite que deux colonnes sur neuf. Les autres ne sont pas comparées : on teste une logique, pas un modèle en entier.

```
dbt test --select test_version_d_epoque
```

Attendu : au vert, en moins d'une seconde. Changez une attente pour voir ce que dbt rend quand le test échoue :

```
actual differs from expected:

@@,numero_commande,cle_concessionnaire
  ,C1             ,ancienne
→ ,C2             ,ancienne→neuve
  ,C3             ,ancienne
```

La flèche montre la ligne fautive, et la colonne donne l'attendu puis l'obtenu. Remettez l'attente juste avant de continuer.

Étape 10 — La documentation

Théorie : `Theorie_05_AE.md`, section 9.

```
dbt docs generate
```

Trois fichiers apparaissent dans `target/`, et il vaut la peine de les peser :

Le fichier	Taille	Ce qu'il porte
`manifest.json`	environ 870 Ko	tout ce que dbt sait du projet, graphe et SQL compilé compris
`catalog.json`	19 Ko	ce que la base sait des objets : colonnes et types
`index.html`	1,7 Mo	le site, qui lit les deux

Ouvrez `target/index.html` dans un navigateur et regardez le graphe de lignage, en bas à droite. Ce que vous voyez n'a été écrit nulle part : dbt l'a déduit de vos `ref()` et de vos `source()`.

© 2026 iAlexMG. Tous droits réservés.

Ouvrez ensuite `manifest.json` dans un éditeur et cherchez `fct_ventes`. Vous y trouverez son SQL compilé, ses dépendances, ses tests, et la description que vous avez écrite dans le YAML. C'est le seul endroit du projet où le lignage existe sous une forme qu'un programme peut lire, et deux cours s'en serviront : le Cours 08, pour qu'un agent sache ce que l'entrepôt contient, et la formation Architecture et Gouvernance, pour remonter d'un rapport jusqu'à la source.

Étape 11 — Déclarer qui consomme

Théorie : `Theorie_05_AE.md`, section 10.

Dans `models/marts/_exposures.yml` :

```yaml
version: 2

exposures:
  - name: tableau_de_bord_des_ventes
    label: "Tableau de bord des ventes"
    type: dashboard
    maturity: high
    url: https://bi.boreal.example/tableaux/ventes
    description: >
      Le tableau de bord mensuel des ventes, construit par l'équipe Business Intelligence
      de Boréal. Il lit l'étoile des ventes et rien d'autre.
    depends_on:
      - ref('rpt_ventes_par_palier')
      - ref('fct_ventes')
      - ref('dim_date')
      - ref('dim_concessionnaire')
      - ref('dim_produit')
    owner:
      name: "Équipe Business Intelligence, Véhicules Boréal inc."
      email: bi@boreal.example
```

Le propriétaire est une équipe et une adresse de groupe, jamais une personne nommée : un nom personnel périme au premier départ, et il transforme un document de projet en annuaire.

Le modèle `rpt_ventes_par_palier` n'existe pas encore, il arrive à l'étape suivante.

Étape 12 — Le résumé du rapport, et l'arbitrage

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_AE.md`, section 11.

Déclarez la couche dans `dbt_project.yml`, sous `models:` :

```
      presentation:
        +materialized: view
        +schema: presentation
```

Puis, dans `models/presentation/rpt_ventes_par_palier.sql` :

```
  select
      d.annee,
      d.mois,
      d.nom_mois,
      c.palier,
      count(*) as lignes,
      sum(f.valeur_nette) as valeur_nette,
      sum(f.marge) as marge

  from {{ ref('fct_ventes') }} as f
  inner join {{ ref('dim_date') }} as d on d.date_jour = f.cle_date
  inner join {{ ref('dim_concessionnaire') }} as c on c.cle_concessionnaire = f.cle_concessionnaire

  group by 1, 2, 3, 4
```

Regardez ce que ce modèle ne fait pas. Il ne porte pas le taux de marge, alors que ce serait une division de deux colonnes qu'il a sous la main. Un taux stocké se moyenne, et une moyenne de taux ne veut rien dire : c'est la démonstration du Cours 04. Le rapport divisera à la lecture, au niveau qu'il affiche.

```
  dbt build
```

```
  select annee, nom_mois, palier, lignes, valeur_nette,
         round(100.0 * marge / valeur_nette, 2) as taux_de_marge
  from presentation.rpt_ventes_par_palier
  where annee = 2025 and mois = 12
  order by palier;
```

Palier	Lignes	Valeur nette	Taux de marge
Argent	423	2 337 121,91 $	28,51 %
Bronze	384	1 678 363,56 $	30,37 %
Or	429	2 596 518,48 $	26,88 %

© 2026 iAlexMG. Tous droits réservés.

Le modèle entier rend 144 lignes, quarante-huit mois par trois paliers.

Écrivez enfin l'arbitrage que vous venez de faire, dans la description du modèle. Trois lignes suffisent, et elles vous éviteront la même discussion dans six mois. Dans `models/presentation/_presentation.yml` :

```yaml
version: 2

models:
  - name: rpt_ventes_par_palier
    description: >
      Le résumé mensuel des ventes par palier d'époque, prêt à servir un rapport.
      Aucun ratio n'y est stocké : le taux de marge se calcule à la lecture, au niveau
      d'agrégation affiché. Les règles de métier vivent dans les MARTS, pas ici.
    columns:
      - name: valeur_nette
        data_tests:
          - not_null
```

Récapitulatif

L'état du projet à la fin du labo :

Ce qui est en place	**Vérification**
Le projet complet	`dbt build --exclude snp_concessionnaire` rend `PASS=35 WARN=0 ERROR=0`
Les tests de données	23, sur les trois dimensions, le fait et le résumé
Le test unitaire	1, sur les trois bornes de la jointure temporelle
Le snapshot	205 lignes, 156 concessionnaires, 49 versions fermées
La dimension en type 2	205 versions, 156 courantes
Le fait sur la version d'époque	75 652 lignes, 430 148 012,16 $, 0,00 $ d'écart
La documentation	`manifest.json` environ 870 Ko, `catalog.json` 19 Ko
L'`exposure`	déclaré au nom de l'équipe de Business Intelligence

© 2026 iAlexMG. Tous droits réservés.

Ce qui est en place	Vérification
La couche `PRESENTATION`	`rpt_ventes_par_palier`, 144 lignes

Le snapshot s'exclut de `dbt build` parce qu'une exécution sans option capture l'état courant, et que l'histoire d'un snapshot se monte dans l'ordre. Une fois les quatre lots rejoués, un `dbt build` complet ne change rien, le dernier lot étant justement l'état courant ; c'est avant qu'il fait des dégâts. Un `dbt build` lancé alors que l'histoire n'est pas montée fait sauter le snapshot au présent, et tout lot ancien rejoué ensuite ouvre une version datée d'un passé, après une version datée d'aujourd'hui. En production, le snapshot tourne à son propre rythme, avant le reste.

Le Cours 06 ouvre les deux étoiles que la vente ne montre pas : l'après-vente et l'usine. Vous y retrouverez la matrice de bus du Cours 02, et les dimensions que vous venez de bâtir y serviront telles quelles.

Pensez à commiter :

```
git add dbt_project.yml models snapshots tests
git commit -m "Cours 05 : les tests, l'historique des paliers, la documentation"
```

Dépannage

Le message ou le symptôme	Ce qu'il veut dire	La parade
`Required var 'lot' not found in config`	un `var()` sans valeur par défaut, et une commande qui compile tout le projet	le défaut du `{% if var('lot', none) %}` de l'étape 6
`Not able to get columns for unit test … because the relation doesn't exist`	une entrée de test unitaire vise un modèle `ephemeral`	donner cette entrée en `format: sql`
Un avertissement de dépréciation à chaque commande	les paramètres d'un test paramétré ne sont pas sous `arguments:`	les y mettre, comme à l'étape 1
`accepted_values` au vert alors que la colonne est vide	le test ignore les valeurs absentes	un `not_null` à côté, toujours
Le fait porte plus de lignes que de couples distincts	l'intervalle de la jointure temporelle est fermé des deux côtés	le rendre semi-ouvert, étape 8
La valeur nette manque de 9,5 millions	des ventes antérieures au premier regard du snapshot ne trouvent aucune version	le plancher de la première version, étape 7

© 2026 iAlexMG. Tous droits réservés.

Le message ou le symptôme	Ce qu'il veut dire	La parade
Le snapshot ajoute une version à chaque exécution	la stratégie `timestamp` sur une colonne qui change à chaque lot	la stratégie `check` sur les colonnes suivies
Les tests restent au vert alors que le staging est cassé	un snapshot est une table, pas une vue	relancer `dbt snapshot` pour que la donnée fausse entre, ou corriger avant

© 2026 iAlexMG. Tous droits réservés.